Lecture 4 revision sheet

Errors, verification & validation

This sheet lists the definitions, formulas and results from Lecture 4, without derivations or discussion. These are in the full notes (`errors-verification-validation.ipynb`, the homework notebook and the supplementary notebook on manufactured solutions); the grey pointers give the section. → L*n* marks material that later lectures build on. [*] marks material that is optional in the notes.

Key messages

1. "Is my model right?" is easy to answer "no" and very hard to answer "yes". A *model* here means everything from the conceptual model to the numerical solution and its post-processing.
2. **Verification**: are we solving the equations correctly? ("Did I build the system right?") **Validation**: are they the right equations for reality and the intended use? ("Did I build the right system?") Verification comes first, and neither guarantees the other.
3. Developers must do **code verification**; every user must do **solution verification** (at least sensitivity testing). Beware getting the right answer for the wrong reasons.
4. The gold standard for code verification is a **convergence test against an exact solution** (analytic, or manufactured): the error must fall at the theoretical rate.
5. Errors are measured with **norms**. The choice of norm matters for outliers and when comparing vectors of different lengths; use RMS or max-norm errors for convergence studies.
6. Round-off, overflow and ill-conditioning mean that small, inevitable errors can sometimes have large consequences. The **condition number** measures how much input errors can be amplified.

1 What is a model, and who uses it?

Full notes: *What is a model; Uses and users of a model*

- A model is a mapping from inputs to outputs. The inputs are: (1) physical data (parameters, forcing, initial and boundary conditions); (2) approximated physical quantities (finite Δt, a mesh, approximated geometry); (3) numerical parameters (discretisation options, stabilisation, solver tolerances); (4) inputs to post-processing and diagnostics. A user can get any of these wrong.
- Whether a model is "right" depends on its **uses and users**: who will use the results, for what, how accurate and how fast they need to be, and whether actual values or only trends are needed.
- Related terms: *digital twin*, *virtual prototyping*, *model–data fusion* (data assimilation), multi-model ensembles. *All models are wrong, but some are useful* (L1).

2 Verification & validation (V&V)

Full notes: *Verification & Validation (V&V); Verification; Validation* → **used in L5, L6, L7**

Verification	(AIAA) "The process of determining that a model implementation accurately represents the developer's conceptual description of the model and the solution to the model." It concerns the maths and the code, and can use artificial, non-physical test problems.
Code verification	Finding and removing bugs. In principle done once, but in practice ongoing and automated, since compilers and libraries change (cf. software quality assurance).
Solution verification	Checking that the numerical error in a particular simulation is acceptable. Needed for every new problem, because of user error and the limits of discretisation. Done by *error estimation* or by *sensitivity testing*: change the user choices (mesh, Δt, scheme; refine *or* coarsen) and check that the conclusions do not change.
Validation	"The process of determining the degree to which a model with its associated data is an accurate representation of the real world as determined by experimental data, the metrics of which are chosen from the perspective of the intended uses of the model." Compares predictions with measurements (laboratory data is a stepping stone to real-world data).
Calibration	Adjusting physical and numerical input parameters to best match data, ideally holding back part of the data to test predictive ability. Leads to data assimilation, inversion, parameter estimation and uncertainty quantification.

- Verification focuses on the software implementation and its application; validation on the conceptual and mathematical models. **Verification is a prerequisite for validation**; verification $\not\Rightarrow$ validation.
- **Right answer, wrong reasons:** one run with a small error proves little. Example: "computing $9x$" as $81/x$ gives a 3.4% error at $x = 2.95$, but fails a simple *trend check* (the output falls as x rises).
- **Sources of numerical error:** round-off (floating-point arithmetic), iterative-convergence error (finite solver tolerances), discretisation error.

Code verification testing hierarchy

Full notes: *A code verification testing hierarchy*

1. **Sanity tests** (qualitative): does it look right, e.g. zero in gives zero out?
2. **Trend / conservation tests** (qualitative): expected trends; conserved quantities (e.g. mass).
3. **Benchmark tests** (quantitative): reproduce accepted community results; also compare run times.
4. **Comparison tests**: side-by-side against a trusted external code; switch terms on and off in both to locate a bug.
5. **Exact solutions** (quantitative, the "gold standard"): (a) analytic solutions, or (b) the method of manufactured solutions (§6).

3 Measuring errors: absolute and relative

Full notes: *Errors; Magnitudes*

For a true value x and an approximation x_{approx}:

$$e_{\text{abs}} = |x - x_{\text{approx}}|, \qquad e_{\text{rel}} = \frac{|x - x_{\text{approx}}|}{|x|} = \left|1 - \frac{x_{\text{approx}}}{x}\right|, \qquad e_{\%} = 100\, e_{\text{rel}}.$$

A relative error puts the size in context: $e_{\text{abs}} = 100$ with $x = 10^6$ is a 0.01% error. For vectors and fields we need norms.

4 Norms

Full notes: *Norms; Vector norms; Norm equivalence…; A note on norm notation* → **used in L5, L6, L8**

A **norm** $\|\cdot\|$ assigns a size to every element and satisfies, for all x, y and scalars α: (1) $\|x\| \geq 0$, with $\|x\| = 0$ iff $x = 0$; (2) $\|x + y\| \leq \|x\| + \|y\|$ (triangle inequality); (3) $\|\alpha x\| = |\alpha|\,\|x\|$.

Vector norms ($v \in \mathbb{R}^n$)

$$\|\boldsymbol{v}\|_2 = \left(\sum_{i=1}^{n} v_i^2\right)^{1/2}, \qquad \|\boldsymbol{v}\|_1 = \sum_{i=1}^{n} |v_i|, \qquad \|\boldsymbol{v}\|_\infty = \max_i |v_i|, \qquad \|\boldsymbol{v}\|_p = \left(\sum_{i=1}^{n} |v_i|^p\right)^{1/p} \ (p \geq 1)$$

Two- (Euclidean, "L2"), one- (taxicab, "L1"), max- (infinity) and p-norms [*].

$$\|\boldsymbol{v}\|_\infty \leq \|\boldsymbol{v}\|_2 \leq \|\boldsymbol{v}\|_1 \leq \sqrt{n}\,\|\boldsymbol{v}\|_2 \leq n\,\|\boldsymbol{v}\|_\infty$$

- The ℓ^p norm gives ℓ^1 at $p = 1$ and tends to ℓ^∞ as $p \to \infty$ (the unit "circles" go from a diamond, through a circle at $p = 2$, to a square).
- **Equivalence:** in finite dimensions all norms are equivalent, $c_1\|\boldsymbol{v}\|_b \leq \|\boldsymbol{v}\|_a \leq c_2\|\boldsymbol{v}\|_b$, so convergence to zero in one norm implies it in all, and convergence *rates* agree, *provided* vectors of different lengths are handled correctly (below).
- **Notation:** ℓ^p (sums) is for vectors and sequences; L^p (integrals) is for functions, e.g. $\|f - g\|_{L^2(I)} = \left(\int_I |f - g|^2\, dx\right)^{1/2}$. "L2 norm" of a vector is common but informal.

Norms and loss functions; the choice of norm matters

Full notes: *Aside: linking norms with loss functions; Warning: Norm Choice Matters (Sometimes!)*

Norm of error $\boldsymbol{e}$	Loss function	Behaviour
ℓ^2	MSE $= \frac{1}{n}\sum e_i^2$; RMSE $= \sqrt{\text{MSE}}$	Squares errors, so **sensitive to outliers** ("least squares", `np.polyfit`).
ℓ^1	MAE $= \frac{1}{n}\sum \lvert e_i\rvert$	Linear in errors, so **robust to outliers**.
ℓ^∞	max error $= \max_i \lvert e_i\rvert$	Controls the worst case only; dominated by a single outlier.

Fitting a line by minimising each norm (`scipy.optimize.minimize`; the ℓ^1 and ℓ^∞ costs are not differentiable, so use `method='Nelder-Mead'`) and then corrupting one data point: the ℓ^2 fit is pulled towards the outlier, the ℓ^1 fit hardly moves, and the ℓ^∞ fit moves most. → **used in L5, L8**

Comparing error vectors of different lengths

Full notes: *Example 2 – comparing vectors of differing lengths; RMS (root mean square) norm… [*]* **→ used in L6, L8**

- Halving Δt doubles the length of the vector of errors over all time levels, so the *sum-based* norms grow with n. For forward Euler on the logistic ODE (L1), whose error is first order:

Error measure	$\|\boldsymbol{e}\|_\infty$	$\|\boldsymbol{e}\|_2$	$\|\boldsymbol{e}\|_1$	RMS	MAE
Observed order	1	$\frac{1}{2}$	0 (no convergence)	1	1

- Fix: compare on a common set of times, or use ℓ^∞, or the **normalised** norms

$$\mathrm{RMS} = \Big(\frac{1}{n}\sum_{i=1}^{n} e_i^2\Big)^{1/2} = \frac{\|\boldsymbol{e}\|_2}{\sqrt{n}}, \qquad \mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|e_i| = \frac{\|\boldsymbol{e}\|_1}{n}.$$

  In code: `sl.norm(a - b) / np.sqrt(len(a))`.
- [*] The RMS error is a discrete analogue of the L^2 norm: with N equally spaced (midpoint) samples on an interval I of length $|I|$, $\|f-g\|_{L^2(I)} \approx \sqrt{|I|}\, e_{\mathrm{RMS}}$ (equal when $|I| = 1$). The factor $\sqrt{|I|}$ is the same for every run in a convergence study, so it does not affect the observed order.

5 Code verification against exact solutions

Full notes: *Comparing algorithms against analytic solutions – code verification; A problem with an exact solution to compare against* **→ used in L5, L6, L7**

- Three scenarios: (1) an analytic solution of the full problem (often possible for ODEs); (2) an analytic solution of a simplified problem (common for PDEs; needs a suite of tests covering all terms); (3) a *manufactured* solution of the full problem plus a source term (§6).
- **SymPy** [*] can find exact solutions:

```
t = sym.symbols('t'); y = sym.Function('y')
sol = sym.dsolve(sym.Eq(y(t).diff(t), y(t) + t**3), y(t), ics={y(0): 1})
y_ex = sym.lambdify(t, sol.rhs)                    (sym.Eq, not ==, builds an equation)
```

- Example: $y' = y + t^3$, $y(0) = 1$ has $y(t) = 7e^t - t^3 - 3t^2 - 6t - 6$. It tests a right-hand side that depends on both t and y. Forward Euler converges at first order in both the end-time and RMS errors. (Homework: SymPy also recovers the L1 logistic solution $x = x_0/(x_0 + (1-x_0)e^{-\alpha t})$.)
- Make every run end exactly at T (start from a Δt that divides T and keep halving). Otherwise compare with $y_{\mathrm{ex}}(t_{\mathrm{last}})$ and check separately that T was reached.

Why log–log plots

Full notes: *Plotting errors using a log-log plot*

- If $y = ax^k$ then $\log y = k \log x + \log a$: a straight line of slope k on log–log axes (e.g. $y = 2x^3$ gives slope 3). If the points *approach* a line of slope p as $x \to 0$, then $y = \mathcal{O}(x^p)$. Larger Δt values may curve away (higher-order terms): only the asymptotic regime counts.
- **Local vs global:** forward Euler's Taylor truncation is $\mathcal{O}(\Delta t^2)$ per step, but $T/\Delta t$ steps accumulate to a *global* error of $\mathcal{O}(\Delta t)$, so the observed slope is 1.

6 Method of manufactured solutions (MMS) [*]

Full notes: *supplementary notebook: The Method of Manufactured Solutions (MMS)*

1. Write the problem as $L(y) = 0$, e.g. $L(y) = y' - y$ for a solver of $y' = y$.
2. Choose a **non-trivial** manufactured solution y_{mms} (not, e.g., a constant, and one for which no term of L vanishes or cancels).
3. Compute the residual $S = L(y_{\mathrm{mms}})$ (symbolically, e.g. with SymPy).
4. y_{mms} exactly solves the modified problem $L(y) = S$, i.e. $y' = y + S(t)$: add S as a **source term** and run a convergence test against y_{mms}.

- Example: $y_{\mathrm{mms}} = 7e^t - 3t^2 - 6t - 5 + \cos 3t + \ln(1+t)$ verifies forward Euler at first order.
- Pitfall: simply using $f(t) = y'_{\mathrm{mms}}(t)$ as the right-hand side (e.g. $7e^t - 6t - 6$ for $y = 7e^t - 3t^2 - 6t - 5$) removes the y-dependence and so tests less of the code. Rewriting it as $f = y + 3t^2 - 1$ restores it.
- MMS lets you test convergence down to very small errors without needing an analytic solution, which makes it particularly powerful for PDEs. As with any test, passing shows only that the code has no bug *this test* can detect: a test can show a code is wrong, never prove it right.

7 How things go wrong: round-off, truncation, overflow [*]

Full notes: *How can things go wrong?; Rounding or round-off Errors; Truncation errors; Overflow*

Double precision (`float64`)	64 bits, about 15–16 significant decimal digits.
Machine epsilon	$\varepsilon = 2^{-52} \approx 2.22 \times 10^{-16}$ (`np.finfo(float).eps`): $1 + \varepsilon > 1$ but $1 + \varepsilon/2 = 1$. (`float32`: $\varepsilon \approx 1.19 \times 10^{-7}$.)
Largest / smallest	max $\approx 1.80 \times 10^{308}$; max$+10^{291}$ rounds back to max, but max$+10^{292} =$ `inf`. Smallest normal $\approx 2.23 \times 10^{-308}$; smallest subnormal $\approx 4.9 \times 10^{-324}$ (below this: *underflow* to 0).
Integers	`int32`: $[-2^{31}, 2^{31} - 1] = [-2147483648, 2147483647]$, and $2^{31} - 1$ plus 1 wraps to -2^{31} (overflow). `int64`: max $2^{63} - 1 \approx 9.22 \times 10^{18}$. Python's own `int` has arbitrary precision.

- **Round-off error:** the difference between a real number and its nearest floating-point representation (e.g. 1/3). Individually tiny, but it can accumulate or be amplified.
- **Loss of significance / catastrophic cancellation:** subtracting nearly equal numbers loses leading digits. E.g. $100000 \cdot \frac{1}{3} - 33333$ is wrong in the 11th digit; $(1 - x)^{10}$ expanded as $1 - 10x + 45x^2 - \cdots + x^{10}$ has a 27% relative error at $x = 0.95$ and even the wrong sign at $x = 0.99$. Mathematically equivalent formulas can behave very differently.
- **Truncation error:** from truncating an infinite process (e.g. a Taylor series, L1); closely linked to discretisation error.
- **Round-off vs truncation** (homework): for $f'(x) \approx \frac{f(x+\Delta x) - f(x)}{\Delta x}$, the truncation error $\approx \frac{\Delta x}{2}|f''|$ falls like Δx, while the round-off error $\approx \varepsilon|f|/\Delta x$ grows (erratically) as Δx decreases. They balance at $\Delta x \sim \sqrt{\varepsilon} \approx 10^{-8}$ (for $f = e^x$ at $x = 1$: minimum error $\approx 2.7 \times 10^{-8}$ at $\Delta x \approx 1.3 \times 10^{-8}$). → used in L7

Fibonacci numbers ($F_{i+1} = F_i + F_{i-1}$) as a test of number representations:

Method	First wrong F_n
Python `int` recurrence	none (exact; F_{1000} correct)
NumPy `int32` array	$n = 47$ (overflow; $F_{46} = 1836311903$ is the last that fits)
NumPy `int64` array	$n = 93$ (overflow; $F_{92} \approx 7.5 \times 10^{18}$)
Eigen-decomposition $Q^n = P\Lambda^n P^{-1}$, $Q = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 0 \end{smallmatrix}\right)$, `float32`	$n = 32$ (round-off)
Same, `float64`	$n = 71$ (round-off)
Same, arbitrary precision (`mpmath`, 500 digits)	none up to $n = 1000$

8 Matrix norms and the condition number [*]

Full notes: *Matrix norms; Vector induced matrix norms; The condition number; Ill-conditioned matrices* → used in L5

Matrix norms (A is $m \times n$)

$$\|A\|_F = \left(\sum_{i,j} A_{ij}^2\right)^{1/2}, \quad \|A\|_\infty = \max_i \sum_j |A_{ij}|, \quad \|A\|_1 = \max_j \sum_i |A_{ij}|, \quad \|A\|_2 = \sigma_{\max}(A) = \sqrt{\lambda_{\max}(A^* A)}$$

Frobenius; maximum absolute row sum; maximum absolute column sum; spectral (σ = singular values). For any vector norm, the *induced* matrix norm is $\|A\| = \max_{\boldsymbol{x} \neq 0} \|A\boldsymbol{x}\|/\|\boldsymbol{x}\|$.

- The 1, 2 and ∞ matrix norms are induced by the corresponding vector norms; the Frobenius norm is not induced. `sl.norm(A)` defaults to Frobenius; `sl.norm(A, 2)` is spectral. Example: $A = \left(\begin{smallmatrix} 10 & 2 & 1 \\ 6 & 5 & 4 \\ 1 & 4 & 7 \end{smallmatrix}\right)$ has $\|A\|_F \approx 15.75$, $\|A\|_\infty = 15$, $\|A\|_1 = 17$, $\|A\|_2 \approx 13.79$.
- Geometrically, A maps the unit circle to an ellipse; $\|A\|_2$ is the largest stretch (the semi-major axis) and $1/\|A^{-1}\|_2 = \sigma_{\min}$ the smallest. For symmetric A, $\|A\|_2 = \max|\lambda(A)|$.

Condition number

$$\mathrm{cond}(A) = \|A\|\,\|A^{-1}\| \quad \left(= \frac{\sigma_{\max}}{\sigma_{\min}} \text{ in the 2-norm: } \texttt{np.linalg.cond} \right)$$

$$\frac{\|\delta\boldsymbol{x}\|}{\|\boldsymbol{x}\|} \leq \mathrm{cond}(A)\,\frac{\|\delta\boldsymbol{b}\|}{\|\boldsymbol{b}\|} \quad \text{for } A(\boldsymbol{x} + \delta\boldsymbol{x}) = \boldsymbol{b} + \delta\boldsymbol{b}.$$

Relative error of output $\lesssim$ condition number $\times$ relative error of input. With only round-off, expect a relative error up to about $\mathrm{cond}(A) \times \varepsilon$.

- $\mathrm{cond}(A) \geq 1$; close to 1 is **well-conditioned**; large is **ill-conditioned**; infinite for a singular matrix. The determinant on its own is *not* a reliable measure: $\det(10^{-3} I) = 10^{-3n}$ is tiny, but $\mathrm{cond}(10^{-3} I) = 1$. Libraries

estimate cond cheaply from the LU factorisation.

- Example: $\left(\begin{smallmatrix} 2 & 1 \\ 2 & 1+\epsilon \end{smallmatrix}\right) \boldsymbol{x} = \left(\begin{smallmatrix} 3 \\ 0 \end{smallmatrix}\right)$. At $\epsilon = 0.001$: $\det = 0.002$, $\text{cond} \approx 5000$, $\boldsymbol{x} = (1501.5, -3000)^T$. Changing ϵ to 0.002 (a 0.1% change in one entry) halves the solution to $(751.5, -1500)^T$.
- Classic ill-conditioned matrices (homework): the **Hilbert** matrix $a_{ij} = 1/(i+j-1)$ ($\text{cond} \approx 1.5 \times 10^{10}$ at $n = 8$, $\approx 2 \times 10^{16}$ at $n = 12$, where solving $A\boldsymbol{x} = \boldsymbol{b}$ gives ~35% error); the **Vandermonde** matrix of polynomial interpolation on equally spaced points in $[0, 1]$ ($\text{cond} \approx 1.5 \times 10^7$ at $n = 10$). → **used in L5**

9 Algorithm stability

Full notes: *Algorithm stability* → **used in L6, L7**

An algorithm is **stable** if it damps, rather than amplifies, the inevitable small errors (round-off, truncation) it introduces. This matters in iterative linear algebra and, especially, when time-stepping ODEs and PDEs, where errors must not grow as the solution is propagated forward.